

Wrapper Classes

Bridging Primitives and Objects

CODEHIVE

1. The Concept

Wrapper classes provide a way to use **primitive data types** (int, boolean, etc.) as **Objects**. They "wrap" the primitive value in an object-oriented shell.

Why do we need them?

- **Collections:** Frameworks like ArrayList and HashMap only work with Objects.
- **Utility Methods:** Objects have methods (like `.toString()`) that primitives lack.
- **Generics:** Primitives cannot be used as Type parameters.

2. The Mapping Table

Each primitive has a corresponding class in the `java.lang` package.

Primitive Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

Note: Only **Integer** and **Character** differ significantly in name from their primitive counterparts.

3. Autoboxing in Action

Java can automatically convert between primitives and wrappers. This is called **Autoboxing**.

```
public class Main {  
    public static void main(String[] args) {  
        // Autoboxing: Primitive to Wrapper  
        Integer myInt = 5;  
        Double myDouble = 5.99;  
        Character myChar = 'A';  
  
        System.out.println(myInt);  
        System.out.println(myDouble);  
        System.out.println(myChar);  
    }  
}
```

4. Unboxing: Extracting Values

To get the primitive value back from an object, you can use built-in methods. However, Java often does this for you automatically (Auto-unboxing).

```
Integer myInt = 10;

// Explicit Unboxing
int primitiveInt = myInt.intValue();

// Manual value extraction examples:
System.out.println(myInt.intValue());
System.out.println(myDouble.doubleValue());
System.out.println(myChar.charValue());
```

5. Collections Integration

This is the most frequent use case for Wrapper classes. Using primitives in an ArrayList will cause a compilation error.

// INVALID

```
ArrayList<int> myNumbers = new ArrayList<>();
```

// VALID

```
ArrayList<Integer> myNumbers = new ArrayList<>();
```

6. String Conversion

Wrapper classes contain powerful utility methods to convert data to and from Strings.

```
Integer myInt = 100;  
String myString = myInt.toString();  
  
// Now you can use String methods!  
System.out.println(myString.length()); // Output: 3
```

7. Parsing Numeric Strings

Static methods in the Wrapper classes allow you to convert a String back into its numeric form.

```
String s = "123";

// Convert String to primitive int
int i = Integer.parseInt(s);

// Convert String to Double object
Double d = Double.valueOf("45.67");
```


8. Performance Considerations

While wrappers are powerful, they carry overhead. Objects require more memory and processing power than primitives.

Best Practice:

- Use **primitives** for high-performance calculations and simple variables.
- Use **wrappers** only when necessary (Collections, Nullable values, or API requirements).

9. Summary & Final Tips

- Wrappers allow primitives to be treated as Objects.
- **Character** and **Integer** are the only unusual names.
- Essential for ArrayList, HashSet, and HashMap.
- Provides `.toString()` and `.parse...()` methods.

© 2026 CodeHive Academy | Core Java Fundamentals